

AI-POWERED EMAIL ASSISTANT

Technical Documentation

An intelligent robotic process automation solution that monitors Outlook, extracts action items using a large language model, and creates calendar events, tasks, draft replies, and a categorised document library.

Built with UiPath Studio 2026 and Google Gemini

Version 1.0.0 · August 2026

Contents

Contents	2
1. Overview	4
1.1 Capability summary	4
2. Solution Architecture	5
2.1 Design principles	5
2.2 Component responsibilities	5
2.3 Email processing pipeline	6
2.4 Document intake pipeline	6
3. Technology Stack	9
4. Repository Structure	10
5. Workflow Reference	11
6. Configuration	12
6.1 Mailbox and scheduling	12
6.2 Language model	12
6.3 Extraction behaviour	13
6.4 Documents and reporting	13
7. The AI Extraction Contract	14
7.1 Response structure	14
7.2 Extraction rules	14
8. Design Decisions	16
8.1 Outlook COM automation instead of Microsoft Graph	16
8.2 Declining the standard framework template	16
8.3 Append-only CSV for the audit log	16
8.4 Semantic embeddings for duplicate detection	16
8.5 Three priority levels, collapsed only at the point of writing	17
8.6 Email as instruction, attachments as context	17
9. Reliability and Error Handling	18
9.1 Failure isolation	18
9.2 Retry policy	18
9.3 Idempotency	18
9.4 Input guards	18
10. Results and Validation	19
10.1 Test matrix	19
10.2 Extraction and output	19
10.3 Attachment processing	20
10.4 Document library	21
10.5 Reporting and audit	23
11. Deployment	25
12. Limitations	27

13. Future Work	28
14. Setup Guide.....	29
14.1 Prerequisites	29
14.2 Installation	29
14.3 Operational notes	29

1. Overview

Professionals lose a measurable amount of time each day performing the same mechanical loop: opening an email, reading it, deciding what it requires, and then manually transcribing that decision into a calendar entry or a task list. The work is low value but not optional, and it scales linearly with inbox volume.

This solution automates that loop end to end. It monitors an Outlook mailbox, sends each new message together with its attachments to a large language model, and converts the model's structured response into real artefacts inside Microsoft 365: calendar appointments, prioritised tasks, and draft replies awaiting the user's approval. A parallel intake path applies the same intelligence to documents dropped into a watched folder, filing each one into a categorised library alongside an automatically generated summary.

The system is built entirely from freely available components. It requires no API registration, no administrative approval, and no paid licence.

1.1 Capability summary

Capability	Description
Continuous monitoring	Polls a configured Outlook folder on a fixed interval and processes newly arrived messages automatically.
AI extraction	Sends message body and attachment text to Gemini, receiving a schema-validated JSON response.
Calendar events	Creates Outlook appointments with location and configurable reminder lead time.
Prioritised tasks	Creates Outlook tasks that synchronise to Microsoft To Do, colour-coded by priority.
Draft replies	Composes a contextual reply and saves it to Drafts. Nothing is ever sent automatically.
Attachment intelligence	Extracts text from PDF, Word, Excel, and plain-text attachments and supplies it to the model as context.
Document library	Classifies, renames, and files documents into category folders with a generated summary alongside each.
Folder intake	Applies the full pipeline to documents that arrive outside email, via a watched folder.
Duplicate suppression	Uses semantic embeddings to recognise re-sent or restated tasks and avoid creating them twice.
Bilingual operation	Processes Arabic and English messages, replying in the language of the original.
Audit logging	Appends a structured record of every processed item to a CSV log.
Weekly reporting	Generates and emails a formatted activity report with an AI-written narrative summary.

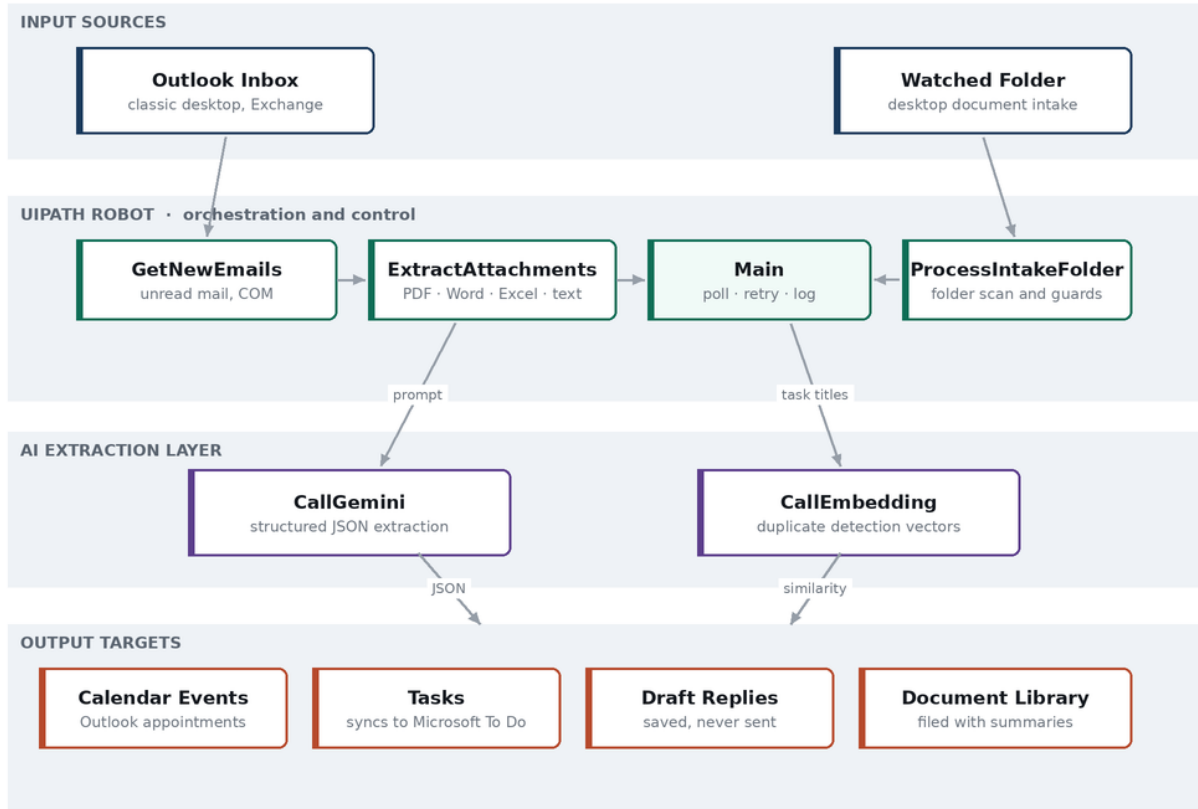
Design intent

Every automated action either creates something reversible or leaves a record. Nothing is sent, deleted, or transmitted outside the user's own mailbox without explicit human action. This constraint shaped several of the decisions documented in Section 8.

2. Solution Architecture

The architecture is deliberately flat. A single orchestrating workflow drives a set of narrowly scoped child workflows, each responsible for one external system or one transformation. There is no queue, no message broker, and no persistent database.

AI-Powered Email Assistant — Solution Architecture



Every component runs locally. The only external dependency is the Gemini API; Outlook is driven through COM automation, requiring no API registration.

Figure 1. End-to-end solution architecture. Two input paths converge on a shared extraction and output pipeline.

2.1 Design principles

- Local execution. Every component runs on the user's own machine. The single external call is to the Gemini API.
- Inherited identity. Outlook is driven through COM automation, so the robot operates as the signed-in user with no credentials of its own.
- Single responsibility per workflow. Each .xaml file addresses one concern and can be tested in isolation.
- Configuration over code. All operational parameters live in a single JSON file; behavioural changes require no rebuild.
- Fail forward. A failure on one item never halts the run; the item is logged and retried on the next cycle.

2.2 Component responsibilities

Layer	Component	Responsibility
Input	Outlook Inbox	Source of incoming messages and attachments.
Input	Watched folder	Source of documents arriving outside email.
Robot	Main	Polling loop, cycle-level error handling, scheduling of the weekly report.
Robot	GetNewEmails	Retrieves unread messages from the configured folder.

Layer	Component	Responsibility
Robot	ExtractAttachments	Saves attachments to disk and extracts their text.
Robot	ProcessIntakeFolder	Scans the watched folder and drives the document pipeline.
AI	CallGemini	Issues the extraction request and returns the model response.
AI	CallEmbedding	Converts task titles into vectors for similarity comparison.
Output	CreateOutlookItems	Creates appointments and tasks through COM.
Output	CreateReplyDraft	Composes and saves an unsent reply.
Output	FileToLibrary	Renames, files, and summarises processed documents.
Output	WriteAuditLog	Appends a structured record of the outcome.

2.3 Email processing pipeline

1. The polling loop retrieves unread messages from the configured Outlook folder, capped at a configurable batch size.
2. For each message, attachments are saved to a temporary directory and their text extracted according to file type.
3. A prompt is assembled from a template, injecting the current date, timezone, sender, subject, body, and extracted attachment text.
4. The prompt is sent to Gemini with a response schema that constrains the reply to a fixed JSON structure.
5. Each extracted task title is embedded and compared against previously created tasks; near-duplicates are skipped.
6. Surviving tasks, meetings, and the reply draft are written to Outlook.
7. Any attachments are classified, renamed, and filed into the document library with a generated summary.
8. The outcome is appended to the audit log, the message is categorised, and it is moved to the processed folder.

Steps two through eight execute inside a per-message error boundary. A failure at any point leaves the message unprocessed and in place, so the next cycle retries it while the remainder of the batch proceeds unaffected.

2.4 Document intake pipeline

The intake path reuses the extraction, output, and logging components without modification. Only the input acquisition and the prompt template differ: there is no sender, no subject, and no reply to compose, so the intake template instructs the model to return an empty reply draft and to treat the document itself as the complete input.

Three guards protect the watched folder from partially written files: an extension filter, a minimum age threshold, and an exclusive-lock test. A per-cycle file cap prevents a large drop from exhausting the API quota in a single pass.

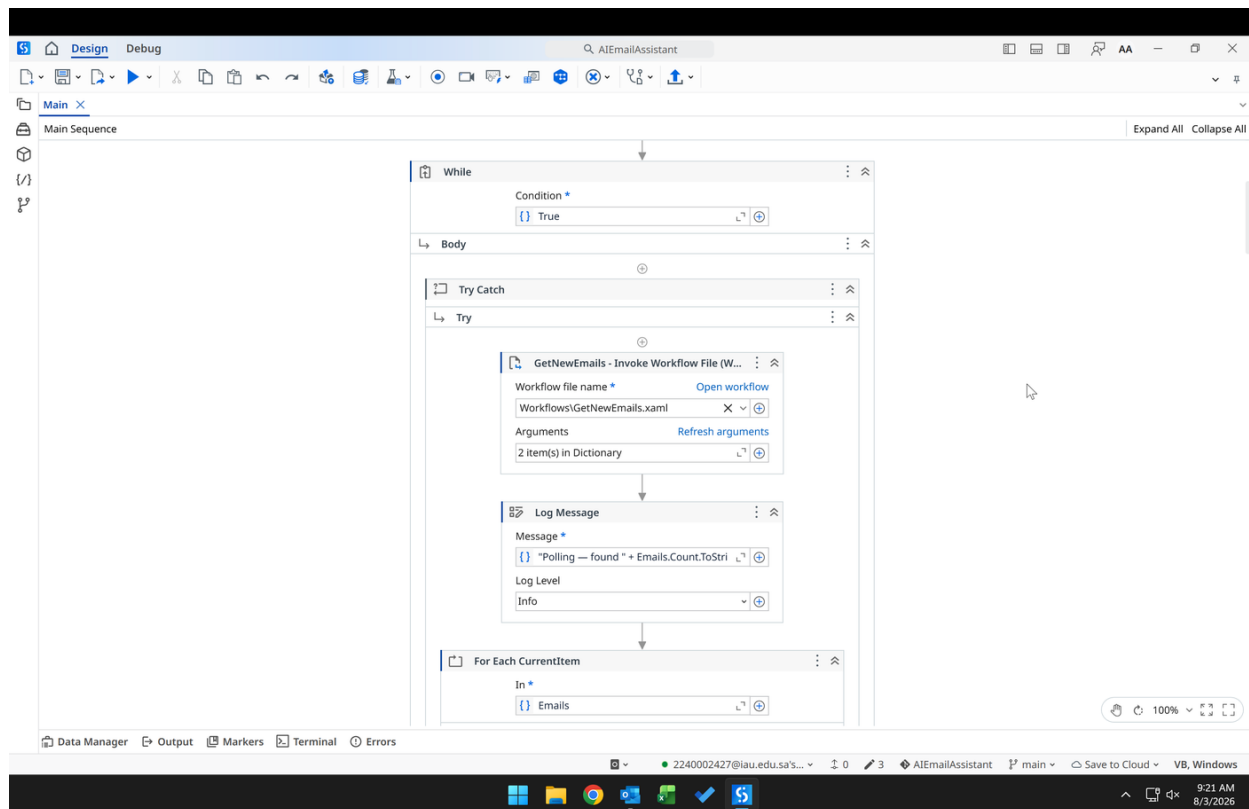


Figure 2. The orchestrating workflow. A perpetual loop wraps a try/catch boundary, with the delay placed in the finally block so the interval is honoured whether the cycle succeeded or failed.

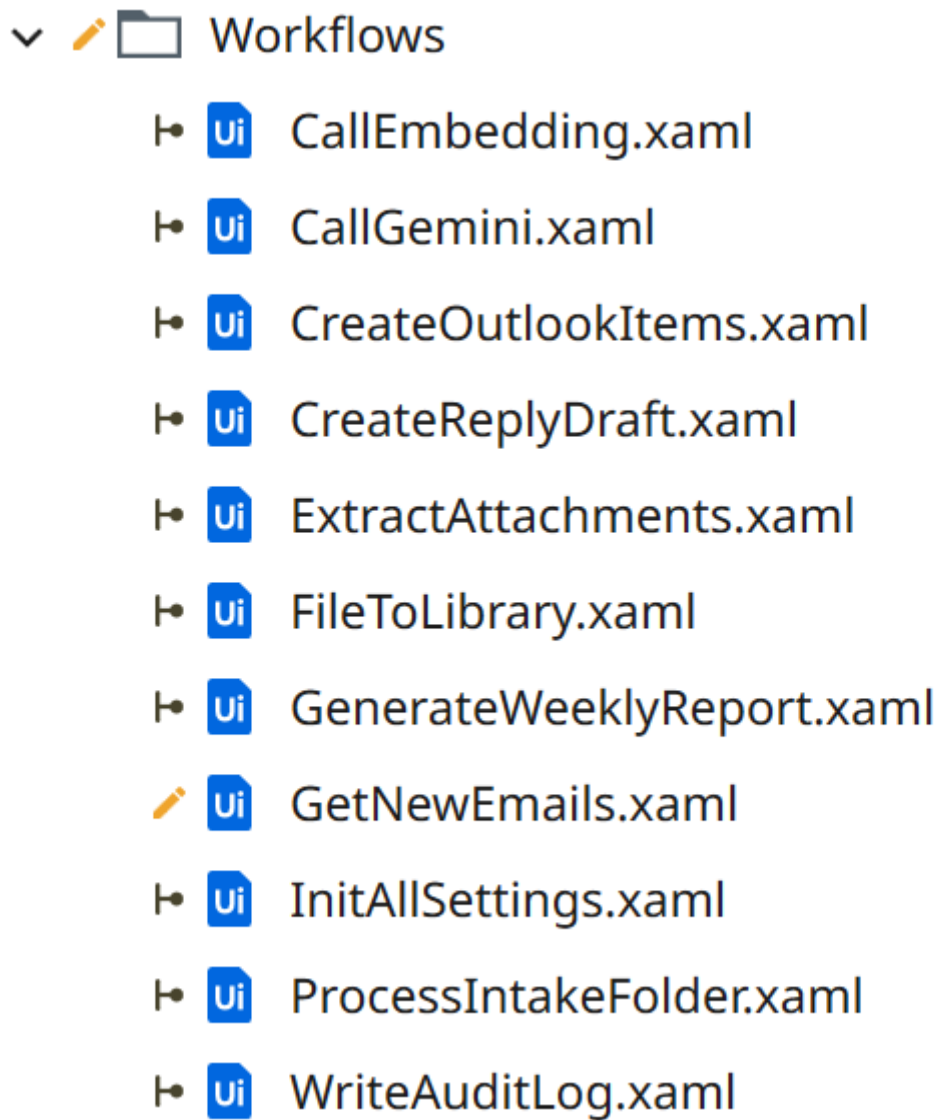


Figure 3. Workflow inventory. Eleven single-purpose files, each independently testable.

3. Technology Stack

Each element of the stack was selected against three constraints: it had to be free, it had to run without administrative privileges, and it had to avoid any dependency on organisational approval.

Component	Selection	Rationale
Automation platform	UiPath Studio 2026 Community Edition	Free for individual use. Native Windows COM interoperability. Visual workflows that a reviewer can follow without reading code.
Language model	Google Gemini (Flash-Lite tier)	Free tier sufficient for the workload. Native support for response schemas, which removes an entire class of parsing failure.
Embedding model	gemini-embedding-001	Free, multilingual, and configurable output dimensionality. Shares authentication with the extraction model.
Mail, calendar, tasks	Outlook desktop via COM	Already authenticated as the user. No API registration required.
Task synchronisation	Outlook Tasks	Tasks created against an Exchange mailbox propagate to Microsoft To Do automatically, with no additional integration.
Document extraction	UiPath PDF activities; Word and Excel COM	Reuses software already installed. No third-party parsing libraries.
Configuration	JSON	Human-readable, trivially parsed, and free of the file-locking behaviour that affects spreadsheet formats.
Audit storage	CSV, append-only	Constant-time appends, resilient to concurrent reads, and openable in any spreadsheet application.
Version control	Git and GitHub	Standard practice. Secrets excluded through .gitignore with a committed configuration template.

4. Repository Structure

AIEmailAssistant/	
├── Main.xaml	Orchestrating polling loop
├── project.json	UiPath project manifest
├── README.md	
├── .gitignore	
├── Config/	
│ ├── Config.json	Runtime settings (git-ignored)
│ ├── Config.template.json	Published template, key redacted
│ ├── PromptTemplate.txt	Email extraction prompt
│ ├── IntakePromptTemplate.txt	Document extraction prompt
│ ├── GeminiRequest.json	Request body with response schema
│ ├── GeminiRequestPlain.json	Request body without schema
│ └── GeminiEmbedRequest.json	Embedding request body
├── Workflows/	
│ ├── InitAllSettings.xaml	Configuration loader
│ ├── GetNewEmails.xaml	Mailbox reader
│ ├── ExtractAttachments.xaml	Attachment text extraction
│ ├── CallGemini.xaml	Extraction API client
│ ├── CallEmbedding.xaml	Embedding API client
│ ├── CreateOutlookItems.xaml	Tasks and appointments
│ ├── CreateReplyDraft.xaml	Reply composition
│ ├── FileToLibrary.xaml	Document classification and filing
│ ├── ProcessIntakeFolder.xaml	Watched-folder pipeline
│ ├── WriteAuditLog.xaml	Structured logging
│ └── GenerateWeeklyReport.xaml	Aggregation and reporting
└── Documentation/	Generated artefacts (git-ignored)

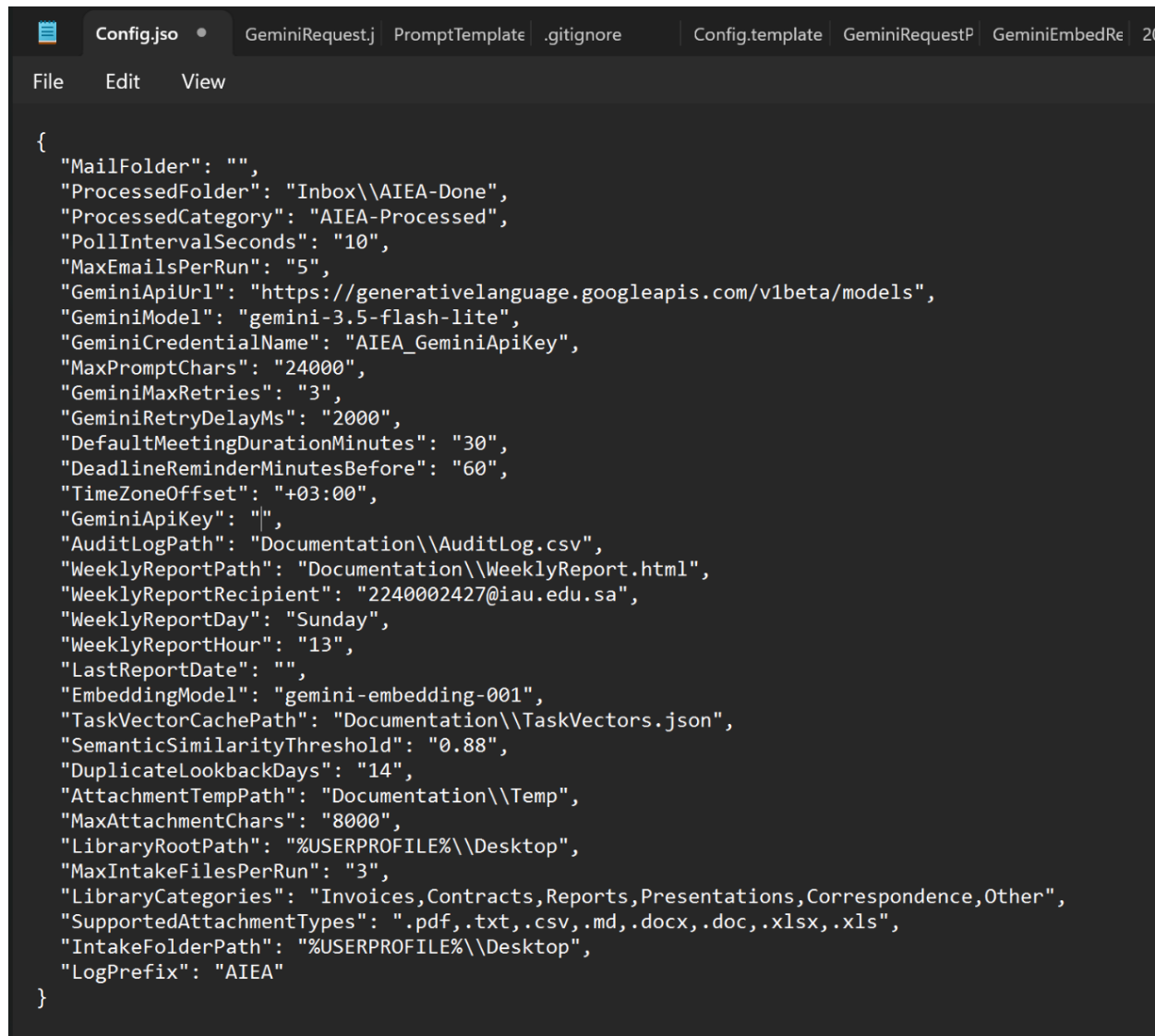
Runtime artefacts are excluded from version control. The audit log and the vector cache contain real sender addresses, subject lines, and document content; the temporary and library directories contain the documents themselves. Only the redacted configuration template is published.

5. Workflow Reference

Workflow	Inputs	Outputs	Notes
InitAllSettings	—	Configuration dictionary	Parses the JSON file and expands environment variables in every value.
GetNewEmails	Config	Message list	Reads unread mail. Does not mark as read; state is tracked by category and folder.
ExtractAttachments	Message, Config	Combined text, file paths	Dispatches by file extension. Office formats are read through COM with guaranteed process cleanup.
CallGemini	Config, prompt, schema flag	Raw response	Truncates oversized prompts, retries transient failures, and distinguishes them from permanent errors.
CallEmbedding	Config, text	Vector	Returns a 768-dimension embedding as a JSON array.
CreateOutlookItems	Extraction, Config	—	Creates tasks and appointments. Performs duplicate checking before each task.
CreateReplyDraft	Address, subject, body	—	Creates an unsent draft. Suppressed for empty bodies and no-reply senders.
FileToLibrary	Extraction, files, Config	—	Moves each document into its category folder and writes a summary file alongside.
ProcessIntakeFolder	Config	—	Scans the watched folder and drives the full pipeline for each qualifying file.
WriteAuditLog	Path, sender, subject, extraction, status	—	Appends a quoted CSV row. Writes a byte-order mark so non-Latin text renders correctly.
GenerateWeeklyReport	Config	Statistics	Aggregates the audit log, requests a narrative, writes a formatted report, and emails it.

6. Configuration

All operational parameters are held in a single JSON file loaded once at startup. Values may contain Windows environment variables, which are expanded during loading so that paths remain portable across machines and user accounts.



```
{
  "MailFolder": "",
  "ProcessedFolder": "Inbox\\AIEA-Done",
  "ProcessedCategory": "AIEA-Processed",
  "PollIntervalSeconds": "10",
  "MaxEmailsPerRun": "5",
  "GeminiApiUrl": "https://generativelanguage.googleapis.com/v1beta/models",
  "GeminiModel": "gemini-3.5-flash-lite",
  "GeminiCredentialName": "AIEA_GeminiApiKey",
  "MaxPromptChars": "24000",
  "GeminiMaxRetries": "3",
  "GeminiRetryDelayMs": "2000",
  "DefaultMeetingDurationMinutes": "30",
  "DeadlineReminderMinutesBefore": "60",
  "TimeZoneOffset": "+03:00",
  "GeminiApiKey": "",
  "AuditLogPath": "Documentation\\AuditLog.csv",
  "WeeklyReportPath": "Documentation\\WeeklyReport.html",
  "WeeklyReportRecipient": "2240002427@iau.edu.sa",
  "WeeklyReportDay": "Sunday",
  "WeeklyReportHour": "13",
  "LastReportDate": "",
  "EmbeddingModel": "gemini-embedding-001",
  "TaskVectorCachePath": "Documentation\\TaskVectors.json",
  "SemanticSimilarityThreshold": "0.88",
  "DuplicateLookbackDays": "14",
  "AttachmentTempPath": "Documentation\\Temp",
  "MaxAttachmentChars": "8000",
  "LibraryRootPath": "%USERPROFILE%\\Desktop",
  "MaxIntakeFilesPerRun": "3",
  "LibraryCategories": "Invoices,Contracts,Reports,Presentations,Correspondence,Other",
  "SupportedAttachmentTypes": ".pdf,.txt,.csv,.md,.docx,.doc,.xlsx,.xls",
  "IntakeFolderPath": "%USERPROFILE%\\Desktop",
  "LogPrefix": "AIEA"
}
```

Figure 4. The complete configuration file. Thirty-three settings govern behaviour without requiring any change to the workflows.

6.1 Mailbox and scheduling

Setting	Purpose
MailFolder	Folder to monitor, relative to the mailbox root. An empty value denotes the Inbox itself.
ProcessedFolder	Destination for successfully processed messages.
ProcessedCategory	Outlook category applied before the message is moved.
PollIntervalSeconds	Delay between cycles.
MaxEmailsPerRun	Upper bound on messages handled in a single cycle.
TimeZoneOffset	Supplied to the model so relative dates resolve correctly.

6.2 Language model

Setting	Purpose
GeminiApiUrl	Base endpoint for both extraction and embedding.
GeminiModel	Extraction model identifier.
EmbeddingModel	Embedding model identifier.
GeminiApiKey	API credential. Excluded from version control.
MaxPromptChars	Prompt ceiling; longer prompts are truncated with an explicit marker.
GeminiMaxRetries	Retry attempts for transient failures.
GeminiRetryDelayMs	Interval between retry attempts.

6.3 Extraction behaviour

Setting	Purpose
DefaultMeetingDurationMinutes	Applied when a meeting has no stated end time.
DeadlineReminderMinutesBefore	Reminder lead time on created appointments.
SemanticSimilarityThreshold	Cosine similarity above which a task is treated as a duplicate.
DuplicateLookbackDays	Age beyond which cached task vectors are discarded.
TaskVectorCachePath	Location of the vector cache.

6.4 Documents and reporting

Setting	Purpose
SupportedAttachmentTypes	Extensions eligible for extraction and filing.
AttachmentTempPath	Staging directory for saved attachments.
MaxAttachmentChars	Ceiling on combined attachment text per message.
LibraryRootPath	Root of the categorised document library.
LibraryCategories	Permitted classification values.
IntakeFolderPath	Watched folder for documents arriving outside email.
MaxIntakeFilesPerRun	Upper bound on files handled in a single cycle.
AuditLogPath	Location of the audit log.
WeeklyReportPath	Output path for the generated report.
WeeklyReportRecipient	Address the report is sent to.
WeeklyReportDay	Weekday on which the report is generated.
WeeklyReportHour	Hour after which generation may occur.

7. The AI Extraction Contract

The interface between the robot and the language model is a strict schema rather than free prose. Gemini accepts a response schema alongside the prompt and guarantees conformance, which eliminates markdown fences, preamble, and structural drift as failure modes. The robot can therefore parse the response with confidence instead of defensively.

7.1 Response structure

```
{
  "email_type": "actionable" | "informational",
  "summary": string, one sentence, under 25 words
  "priority": "High" | "Normal" | "Low",
  "reply_draft": string, empty when no reply is warranted

  "tasks": [ {
    "title": string,
    "due_date": string, ISO 8601, or empty
    "notes": string,
    "priority": "High" | "Normal" | "Low"
  } ],

  "meetings": [ {
    "title": string,
    "start": string, ISO 8601
    "end": string, ISO 8601
    "location": string
  } ],

  "documents": [ {
    "filename": string,
    "category": string, one of the configured categories,
    "suggested_name": string, descriptive, no extension
    "summary": string, two to three sentences
    "key_dates": string, labelled, comma separated
  } ]
}
```

7.2 Extraction rules

The prompt encodes twenty numbered rules. The following carry the most weight in practice, each having been added in response to an observed failure.

Concern	Rule	Failure it prevents
Ownership	Extract only actions the recipient must personally perform.	Tasks generated from work explicitly assigned to other people.
Source hierarchy	The email body is the instruction; attachments are context only.	A contract's own clauses becoming tasks on the reader's list.
Date grounding	Resolve relative dates against today's date; never output a past date.	Weekday names resolving backwards into the previous week.
Meeting certainty	Create a meeting only for one unambiguous slot; otherwise create a scheduling task.	The model inventing a specific time from a range of options.
Priority independence	Judge each task's priority separately from the message's overall priority.	Every task inheriting the urgency of the most urgent one.
Actionable dating	High-priority tasks always receive a due date.	Urgent items sinking to the bottom of an undated task list.
Language parity	Reply in the language of the original; normalise numerals.	Arabic messages producing English tasks.

Concern	Rule	Failure it prevents
Graceful degradation	If attachment text is missing or unreadable, work from the body alone.	The model reporting extraction errors as user-facing content.

Determinism

The extraction request sets temperature to zero. Identical input therefore produces identical output, which makes both duplicate detection and demonstration behaviour predictable. The weekly narrative uses a higher temperature, since prose benefits from variation where structured extraction does not.

Output

Source Debug

AIEmailAssistant execution started

Config loaded: 33 settings. Folder= | Model=gemini-3.5-flash-lite

Polling — found 1

Created task: "Submit the Q4 supplier audit checklist" — best 0.708 vs "Complete and send Q3 budget review section"

Created task: "Refresh the onboarding slides for the new analysts" — best 0.670 vs "Update the process documentation for the intake workflow"

Created task: "Take a look at the conference agenda Fatima circulated" — best 0.779 vs "Have a look at the training materials Fatima shared"

EXTRACTED: {

"email_type": "actionable",

"summary": "Omar asks you to submit the overdue Q4 supplier audit checklist tomorrow and refresh onboarding slides later this month.",

"priority": "High",

"reply_draft": "Salam Omar, I have noted the tasks. I will submit the Q4 supplier audit checklist by tomorrow and take care of the onboarding slides before the end of the month...."

"tasks": [

{

"title": "Submit the Q4 supplier audit checklist",

"due_date": "2026-08-04T17:00:00",

"notes": "Urgent, overdue item.",

"priority": "High"

},

{

"title": "Refresh the onboarding slides for the new analysts",

"due_date": "",

"notes": "To be completed before the end of the month.",

"priority": "Normal"

},

{

"title": "Take a look at the conference agenda Fatima circulated",

"due_date": "",

"notes": "No rush on this one.",

"priority": "Low"

}

}]

,

..

..

..

Figure 5. A single message yielding three tasks at distinct priorities, with due dates resolved from natural language. Similarity scores against previously created tasks are shown above the extraction.

15

Public - عام

8. Design Decisions

The following decisions shaped the solution more than any individual feature. Each is recorded with the constraint that produced it and the cost that was accepted.

8.1 Outlook COM automation instead of Microsoft Graph

Context. Microsoft Graph is the conventional route to Outlook mail, calendar, and tasks. It requires an application registration in Azure Active Directory, which was not available.

Decision. Drive the installed Outlook desktop application directly through COM automation.

Rationale. Outlook is already authenticated as the signed-in user, so the robot inherits that identity without holding any credential of its own. This removed the registration dependency entirely. It also unlocked the task requirement: tasks created against an Exchange mailbox propagate to Microsoft To Do automatically, so no separate integration was needed.

Trade-off. The solution is bound to Windows with classic Outlook installed and running. It cannot execute headless or on a server without a mail profile.

8.2 Declining the standard framework template

Context. UiPath's Robotic Enterprise Framework is the conventional starting point for production automation and would have been the expected choice.

Decision. Build a modular sequence project instead, with a configuration file, one workflow per responsibility, and explicit error boundaries.

Rationale. That framework exists to consume Orchestrator queues transactionally. This solution has no queue: its work arrives as mail and files, and its state lives in Outlook. Adopting the template would have added several layers of state-machine ceremony around logic that does not need them, at a cost in both build time and reviewer comprehension.

Trade-off. The project cannot be dropped into a queue-based orchestration model without restructuring. Given that unattended execution is outside the current scope, this cost is deferred rather than incurred.

8.3 Append-only CSV for the audit log

Context. The audit log is written on every processed item while the robot runs continuously, and it is read by the weekly report.

Decision. Write the log as an append-only CSV with explicit quoting and a byte-order mark, and generate the formatted report as HTML.

Rationale. A spreadsheet write requires opening, reading, and rewriting the entire workbook on every append, and fails outright if the file is open for inspection. A CSV append is a constant-time operation that tolerates concurrent reads. Separating capture from presentation means the log stays robust while the report stays readable.

Trade-off. The log carries no formatting or formulas. This is addressed by the reporting layer rather than the capture layer.

8.4 Semantic embeddings for duplicate detection

Context. Re-sent or restated messages produce tasks with the same meaning but different wording. A first implementation compared word overlap between task titles.

Decision. Replace lexical comparison with cosine similarity over sentence embeddings, cached locally so that only new tasks require an API call.

Rationale. Word overlap compares spelling, not meaning. It missed genuine duplicates phrased differently, and it flagged distinct tasks that shared a noun phrase: "update the deck" and "review the deck" scored above the duplicate threshold despite being different work. Embeddings capture semantic distance, which is the property the question actually depends on. The multilingual model additionally makes detection work across languages at no extra cost.

Trade-off. One additional API call per new task. The threshold is tuned conservatively because the two error types are not symmetric: a false positive silently suppresses a real task, whereas a false negative produces a visible duplicate that takes seconds to dismiss.

8.5 Three priority levels, collapsed only at the point of writing

Context. Microsoft To Do renders importance as a binary flag, so a three-level classification cannot be displayed directly.

Decision. Retain three levels throughout extraction, logging, and reporting; collapse to binary only in the Outlook write. Low-priority tasks are additionally left undated so they sort naturally to the bottom of the list.

Rationale. Collapsing early would have destroyed a distinction the audit log and weekly report both use. Collapsing late costs nothing and preserves the richer signal for every consumer that can represent it.

Trade-off. The Low and Normal distinction is not visible in the task interface itself, and is conveyed indirectly through the presence or absence of a due date.

8.6 Email as instruction, attachments as context

Context. Once attachment text was supplied to the model, a document's own contents became eligible for extraction. A contract states obligations in imperative language throughout, almost none of which belong to the reader.

Decision. Establish an explicit hierarchy in the prompt: the message body is the instruction, and attachments may only inform the detail of tasks the body already implies.

Rationale. Without this rule the system extracted supplier obligations from contract clauses and produced them as personal tasks. The hierarchy converts the attachment from a second source of instructions into a source of supporting detail, which is what makes attachment processing safe rather than merely capable.

Trade-off. Genuinely actionable items that appear only inside an attachment, with no corresponding request in the message body, will not be extracted.

9. Reliability and Error Handling

9.1 Failure isolation

Error handling operates at two levels. A per-item boundary surrounds the processing of each individual message or document; a per-cycle boundary surrounds the batch. A failure inside the inner boundary is logged, the item is left in place, and processing continues with the next item. A failure that escapes to the outer boundary is logged and the loop proceeds to the next cycle.

Because a failed message is neither categorised nor moved, it is retried automatically on the following cycle. Persistent failures therefore appear as repeated entries for the same subject in the audit log, which is the intended signal for human investigation.

9.2 Retry policy

The extraction call distinguishes transient failures from permanent ones. Rate limiting and server-side errors are raised as exceptions inside a retry scope and reattempted with a fixed delay. Authentication failures and malformed requests are logged and allowed to fail immediately, since no number of retries will resolve them. This distinction avoids consuming the retry budget on conditions that cannot improve.

9.3 Idempotency

Two independent mechanisms prevent repeated work. At the message level, successful processing applies a category and moves the message out of the monitored folder, so it cannot be selected again. At the task level, semantic comparison against recently created tasks suppresses near-duplicates even when they arrive from a different message.

9.4 Input guards

Guard	Protects against
Prompt truncation with an explicit marker	Oversized input exceeding model limits, and the model summarising content it only partially received.
Separate attachment text ceiling	A long document displacing the message body from the prompt.
Batch size caps on both input paths	A large backlog exhausting the API quota in a single cycle.
Extension allow-list	Unsupported binary formats reaching the extraction stage.
Minimum file age and exclusive-lock test	Files still being written or copied being read prematurely.
Guaranteed COM process termination	Orphaned Word and Excel processes accumulating over long runs.
No-reply sender detection	Draft replies addressed to unmonitored automated senders.

10. Results and Validation

Validation used a fixed set of messages, each constructed to exercise a specific behaviour, together with two representative documents: a one-page operational status report and a one-page managed services agreement.

10.1 Test matrix

Scenario	Expected behaviour	Result
Single action with a hard deadline	One task, deadline resolved from natural language	Pass
Meeting with an explicit weekday and time	One appointment; no task	Pass
Purely informational newsletter	No task, no appointment, no draft reply	Pass
Multiple actions of mixed urgency	Three tasks at distinct priorities; work assigned to a third party excluded	Pass
Ambiguous proposed timing	No appointment; a scheduling task instead	Pass
Arabic message	Task and reply produced in Arabic; numerals normalised	Pass
Message with a report attached	Task derived from the request, informed by document content	Pass
Message with a contract attached	Signature task only; supplier obligations excluded	Pass
Document dropped into the watched folder	Full pipeline applied; document filed with summary	Pass
Repeated message	Near-duplicate tasks suppressed	Pass
Invalid API credential	Failure classified as permanent; retries not consumed	Pass
Scheduled weekly report	Generated once, then suppressed for the remainder of the day	Pass

10.2 Extraction and output

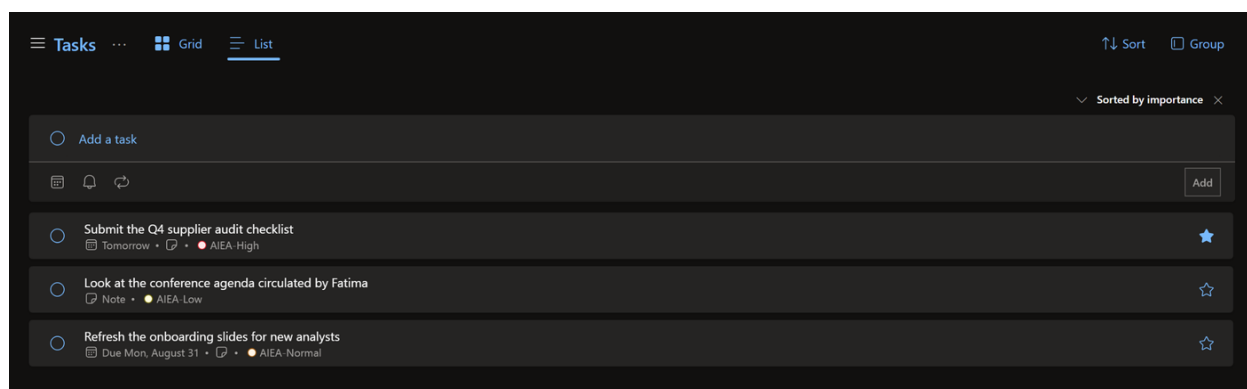


Figure 6. Extracted tasks in Microsoft To Do. Three priorities are distinguished by category colour; only the high-priority item is flagged, and the low-priority item is deliberately undated.

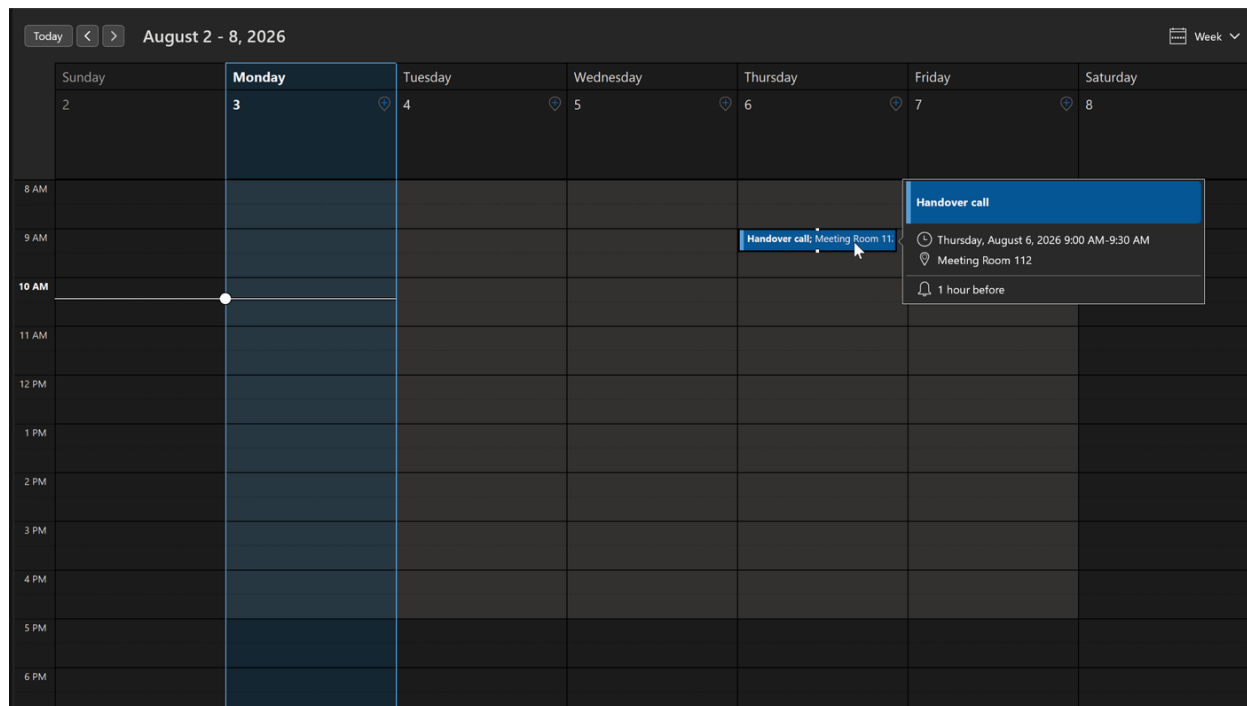


Figure 7. An appointment created from a weekday and time stated in natural language, with location and reminder applied from configuration.

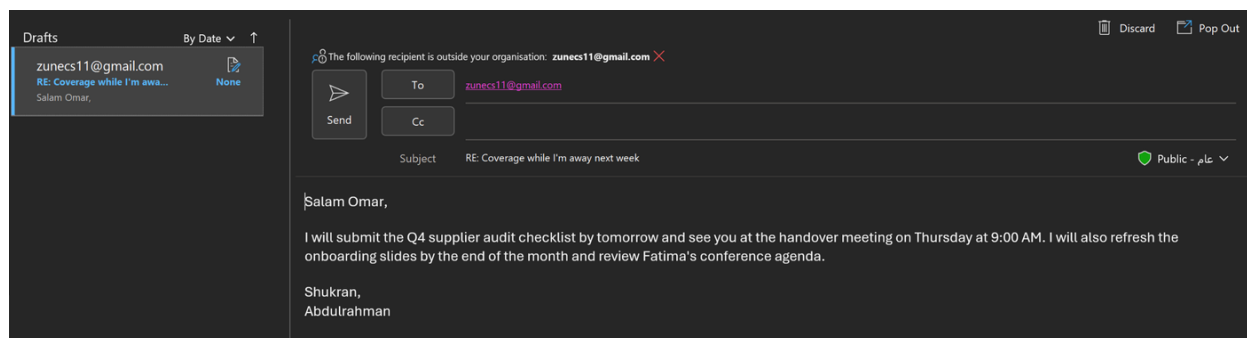


Figure 8. A generated reply saved to Drafts. The draft commits only to actions stated in the original message and is never sent automatically.

10.3 Attachment processing

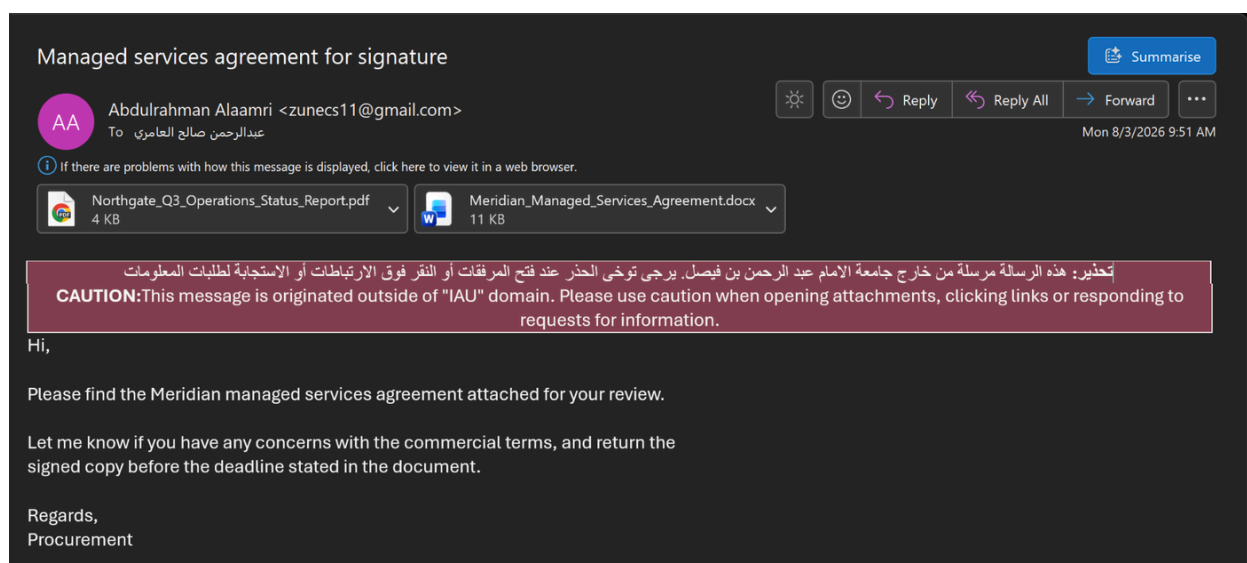


Figure 9. Input message carrying two attachments in different formats, exercising both the PDF and the Word extraction paths in a single run.

Output

Source: Debug

```

AIEmailAssistant execution started
Config loaded: 33 settings. Folder= | Model=gemini-3.5-flash-lite
Polling — found 1
{"activityName":"UiPath.PDF.Activities.PDF.ReadPDFText","org":"43976ca2-7513-42ce-bf3a-64b4298392db","tenant":"c80899d6-f214-4862-8fca-558e15a16d24","user":"224000242..."}
Created task: "Review and return signed Meridian managed services agreement" — best 0.609 vs "Complete and send Q3 budget review section"
EXTRACTED: {
  "email_type": "actionable",
  "summary": "Abdulrahman Alaaamri sent the Meridian managed services agreement for review and signature before August 14, 2026.",
  "priority": "High",
  "reply_draft": "Hi Abdulrahman,\n\nI have received the Meridian managed services agreement and will review the commercial terms before returning the signed copy by August...",
  "tasks": [
    {
      "title": "Review and return signed Meridian managed services agreement",
      "due_date": "2026-08-14T23:59:59",
      "notes": "Review commercial terms of agreement reference MSL-2026-0417 and return signed copy before the deadline.",
      "priority": "High"
    }
  ],
  "meetings": [],
  "documents": [
    {
      "filename": "20260803095327_Northgate_Q3_Operations_Status_Report.pdf",
      "category": "Reports",
      "suggested_name": "Northgate_OperationsStatusReport_Q32026",
      "summary": "This report details the Q3 operational performance of Northgate Logistics Distribution and Fleet Division, highlighting throughput increases, KPI tracking, fleet...",
      "key_dates": "reported 2026-04-01, issued 2026-07-26, pilot 2026-05-20, updated 2026-09-30, budget 2026-10-15"
    },
    {
      "filename": "20260803095327_Meridian_Managed_Services_Agreement.docx",
      "category": "Contracts",
      "suggested_name": "Meridian_ManagedServicesAgreement_MSL20260417",
      "summary": "This agreement outlines the terms between Northgate Logistics and Meridian Systems LLC for fleet telematics and maintenance scheduling platform services..."
    }
  ]
}

```

Figure 10. The corresponding extraction. A single task derives from the message's own request; the contract's internal obligations are correctly excluded. Both documents are classified independently with generated names, summaries, and key dates.

10.4 Document library

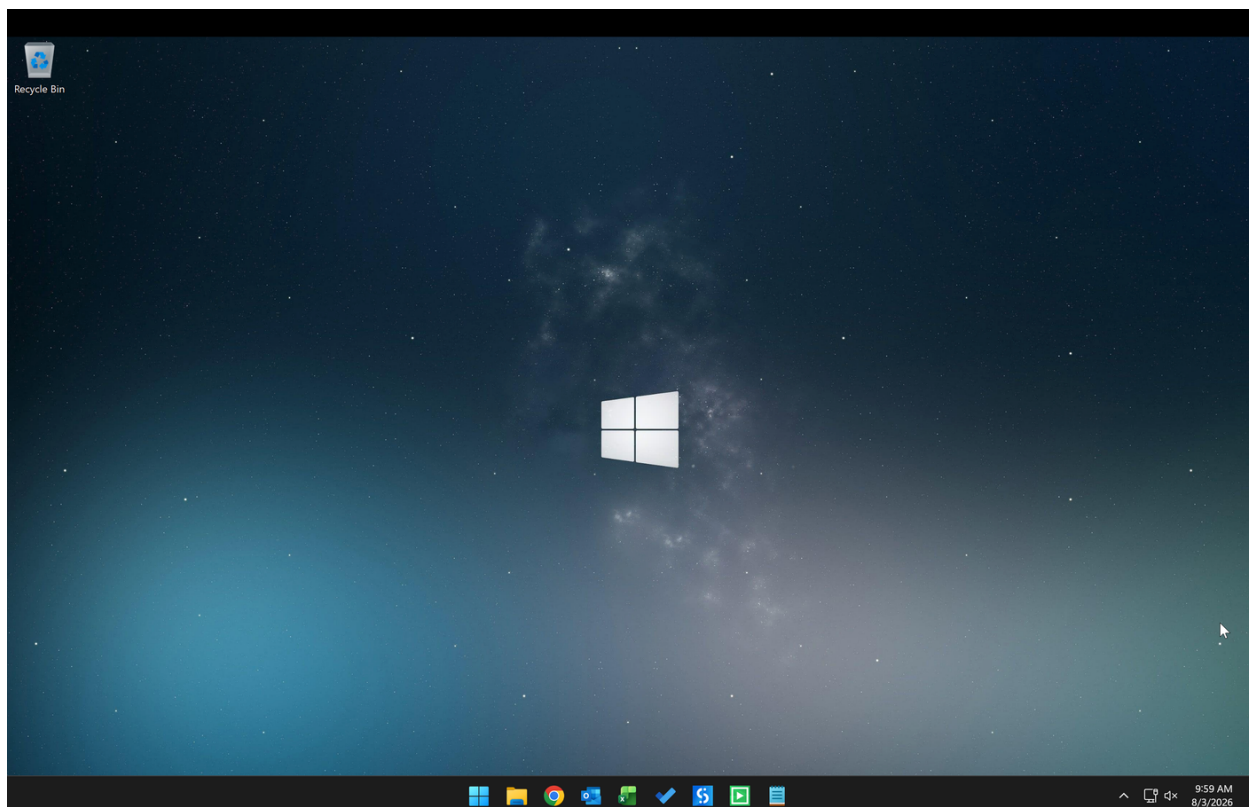


Figure 11. Watched folder before processing.

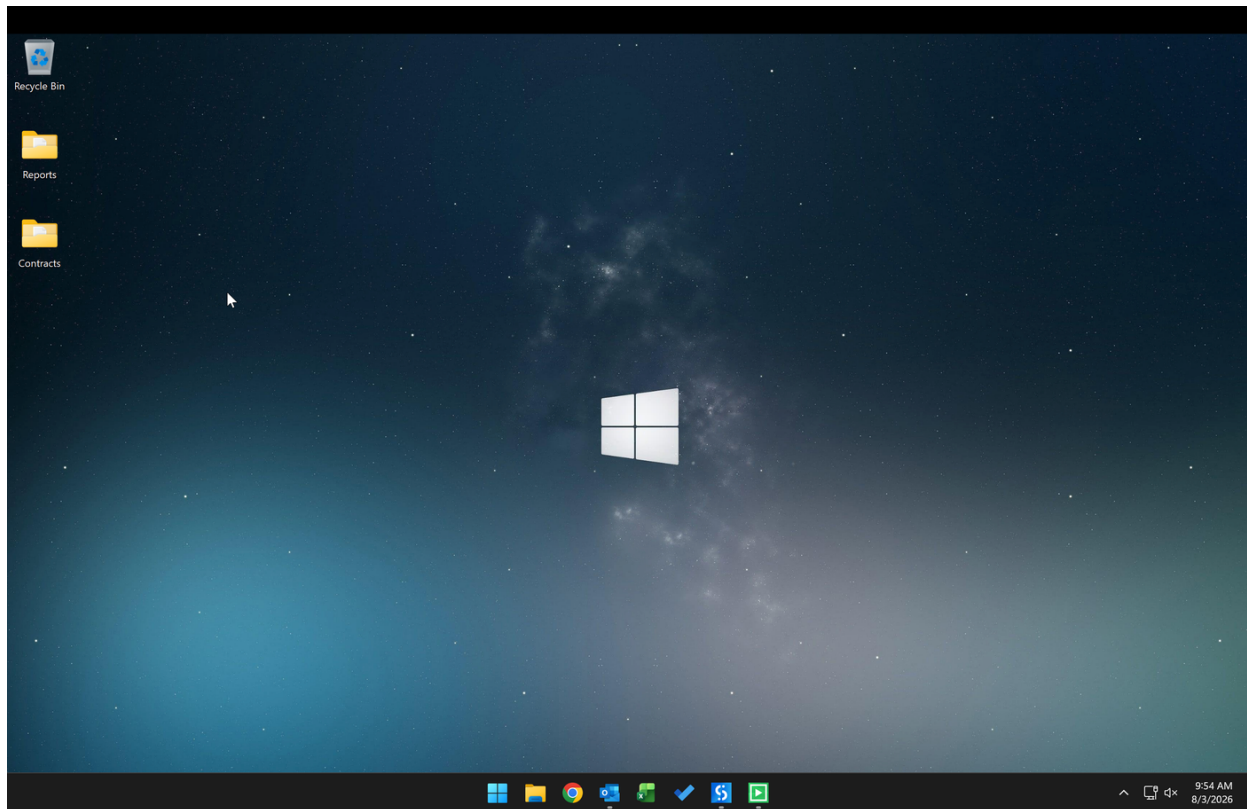


Figure 12. After processing. Category folders have been created and the source files moved into them.

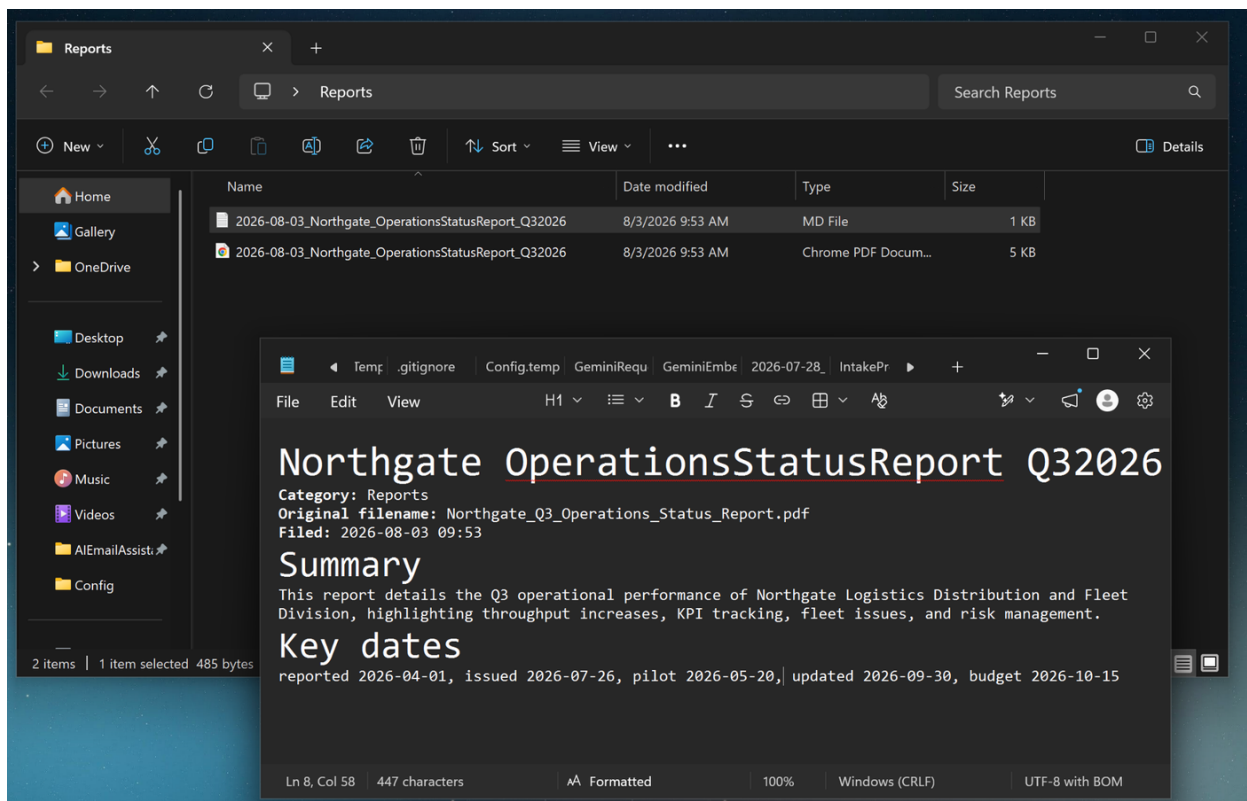


Figure 13. A filed report with its generated summary. Dates have been extracted from within the document body, not from the message that carried it.

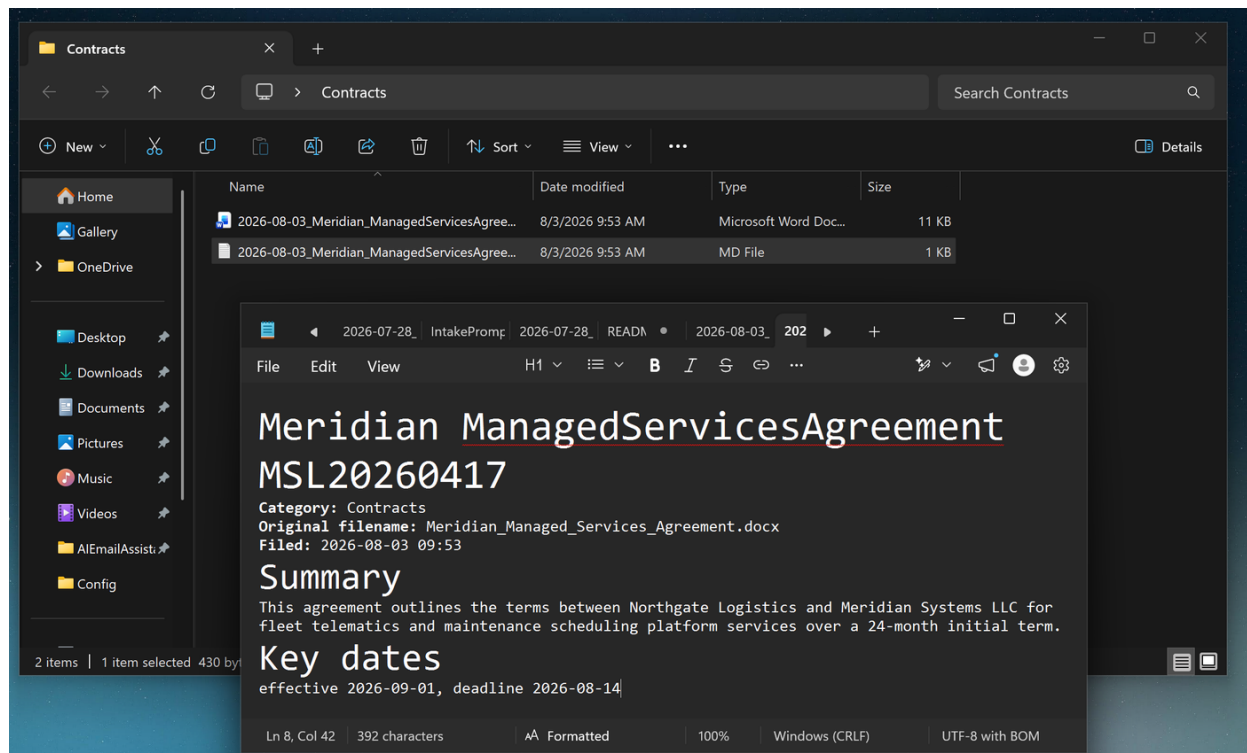


Figure 14. A filed agreement. The effective date and the signature deadline were both recovered from the document's own contents.

10.5 Reporting and audit

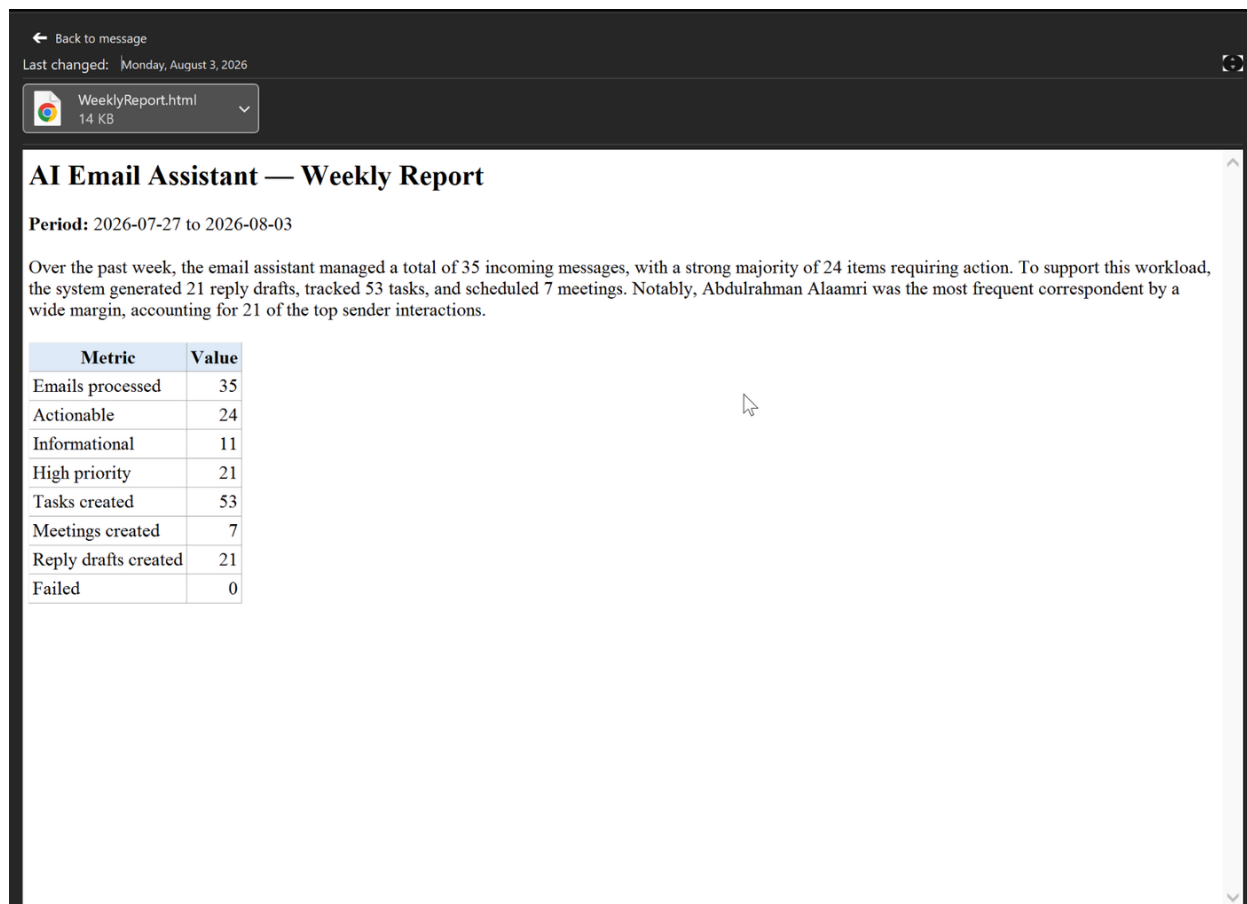


Figure 15. The generated weekly report. Figures are computed from the audit log; the narrative paragraph is written by the model from those figures alone.

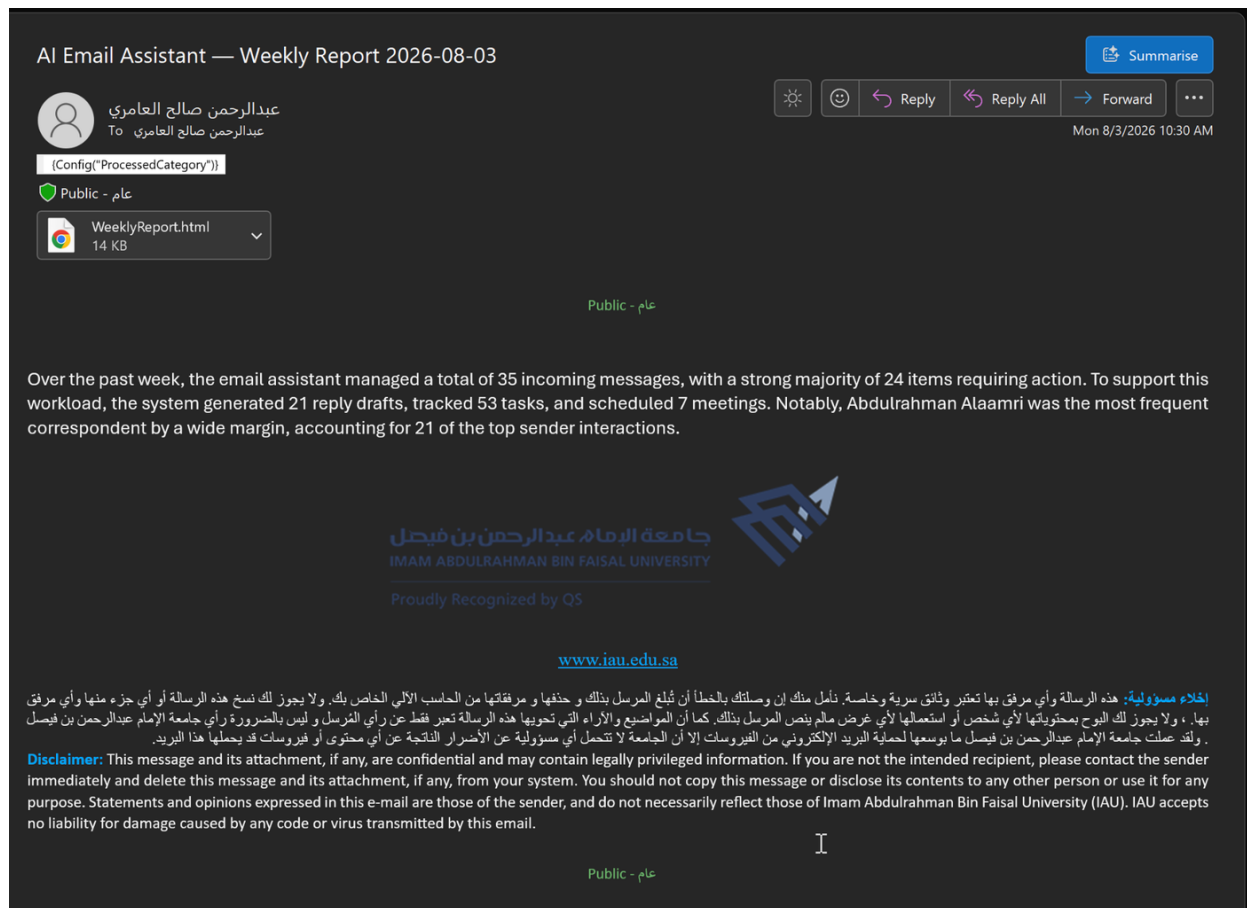


Figure 16. The report delivered as an email attachment.

Timestamp	Sender	Subject	Email Type	Priority	Tasks	Creat	Meetings	Cr	Status	Reply	Drafts	Summary
1	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	shared	three handover items before his travel, including urgent vendor invoice reconciliation.
2	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	sent	a list of three handover items before his travel, including urgent vendor invoice reconciliation.
3	Abdulrahman Al-Aamri	IT Newsletter	informational	Low	0	0	0	SUCCESS	No	The July IT	newsletter is available on the intranet portal, highlighting updates like the new VPN client rollout.	
4	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	sent	three handover items, including an urgent invoice reconciliation due tomorrow and documentation updates.
5	Abdulrahman Al-Aamri	Quick sync	actionable	High	1	1	1	SUCCESS	Yes	Layla	requested	a 30-minute sync next week regarding the automation roadmap and an updated cost estimate sent ASAP.
6	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and meet on Tuesday at 10:00 AM.	
7	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and attend an alignment meeting on Tuesday.	
8	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review section by Wednesday and meet on Tuesday.	
9	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	3	0	0	SUCCESS	Yes	Omar	sent	three handover items before his travel, including an urgent invoice reconciliation due tomorrow.
10	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and meet on Tuesday.	
11	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and meet on Tuesday.	
12	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and meet on Tuesday.	
13	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review section by Wednesday at 12:00 noon and meet on Tuesday.	
14	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	requests	help with three handover items before his travel, including urgent vendor invoice reconciliation.
15	Abdulrahman Al-Aamri	Handover it	actionable	High	1	1	1	SUCCESS	Yes	Omar	requested	completion of the overdue vendor invoice reconciliation, process documentation update, and review of Fatima's training materials.
16	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Submit	your Q3 budget review deck section by Wednesday noon and meet on Tuesday at 10:00 AM.	
17	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Submit	your Q3 budget review section by Wednesday at 12:00 noon and meet on Tuesday.	
18	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	sent	three handover items before his travel, including an urgent vendor invoice reconciliation.
19	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	asked	Abdulrahman to complete the vendor invoice reconciliation, update the process documentation, and review training materials before his travel.
20	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	shared	three handover items including vendor invoice reconciliation, process documentation updates, and training materials review.
21	Abdulrahman Al-Aamri	Q3 Budget	actionable	High	1	1	1	SUCCESS	Yes	Submit	your Q3 budget review section by Wednesday and meet on Tuesday.	
22	22400024	TEST	informational	Low	0	0	0	SUCCESS	No	This is a	test email sent from Imam Abdulrahman Bin Faisal University containing standard disclaimers.	
23	22400024	TEST	informational	Low	0	0	0	SUCCESS	No	This is a	test email sent from Abdulrahman Al-Aamri with no actionable items or content.	
24	Abdulrahman Al-Aamri	Please review	actionable	High	1	0	0	SUCCESS	Yes	Abdulrahman	sent	his progress report and asked to review the challenges section before Thursday's meeting.
25	Abdulrahman Al-Aamri	Please review	actionable	High	1	0	0	SUCCESS	Yes	Abdulrahman	sent	his progress report and asked you to review the challenges section before Thursday's meeting.
26	Document Som	Random	actionable	Normal	3	0	0	SUCCESS	No	Biweekly	progress report detailing automation tasks, challenges with UIPath and translation tokens, and ongoing development activities at S.	
27	Document Som	somerando	actionable	Normal	3	0	0	SUCCESS	No	Biweekly	progress report detailing automation tasks, challenges, and training activities at Saudi Aramco.	
28	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	requests	vendor invoice reconciliation closure, intake workflow process documentation updates, and reviewing Fatima's training materials.
29	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	requests	vendor invoice reconciliation closure, intake workflow process documentation updates, and reviewing Fatima's training materials.
30	Abdulrahman Al-Aamri	Handover it	actionable	High	3	0	0	SUCCESS	Yes	Omar	requests	vendor invoice reconciliation closure, intake workflow process documentation updates, and reviewing Fatima's training materials.
31	Institute	المؤسسة	informational	Low	0	0	0	SUCCESS	No	المؤسسة	المؤسسة	المؤسسة

Figure 17. The audit log. Every processed item is recorded with its classification, counts, outcome, and summary. Arabic entries render correctly owing to the byte-order mark written on each append.

11. Deployment

The solution is published as a versioned package to a personal workspace and runs from UiPath Assistant, independently of the development environment. Configuration travels with the package, so the deployed instance is self-contained.

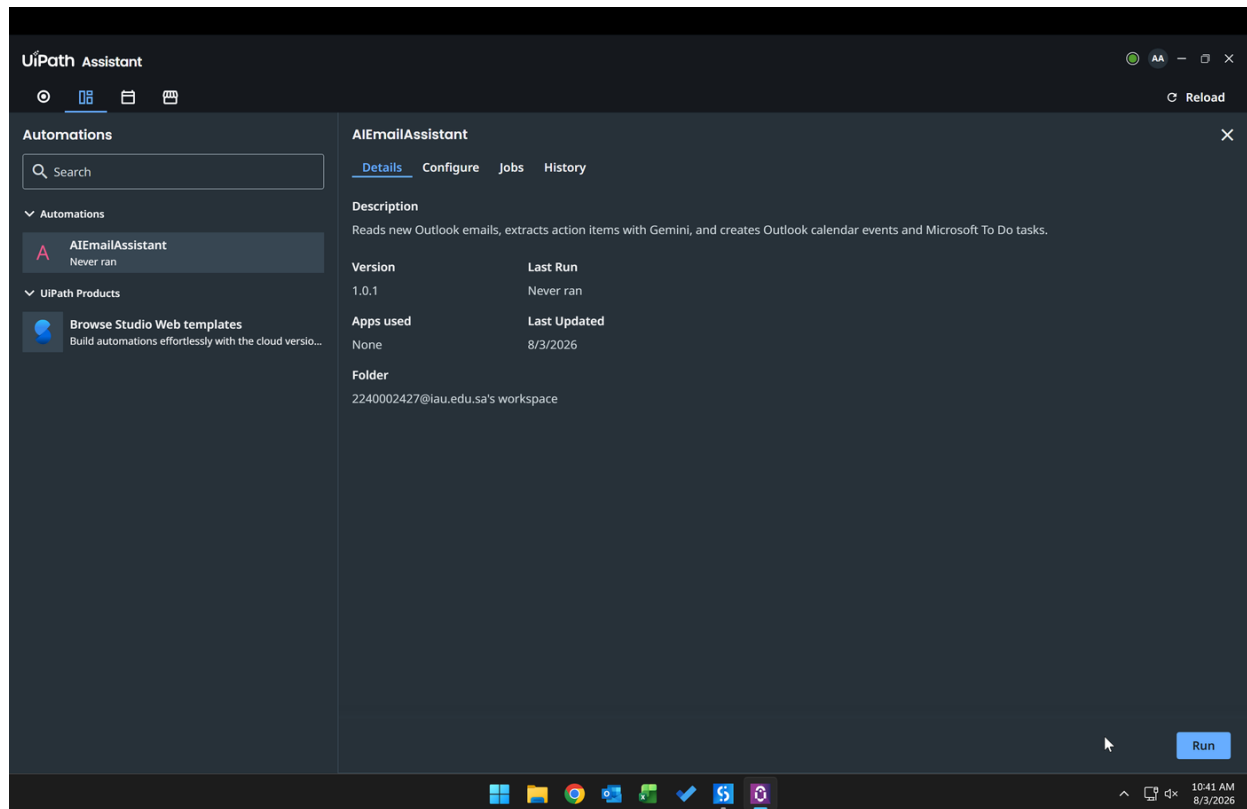


Figure 18. The published automation available for execution outside the development environment.

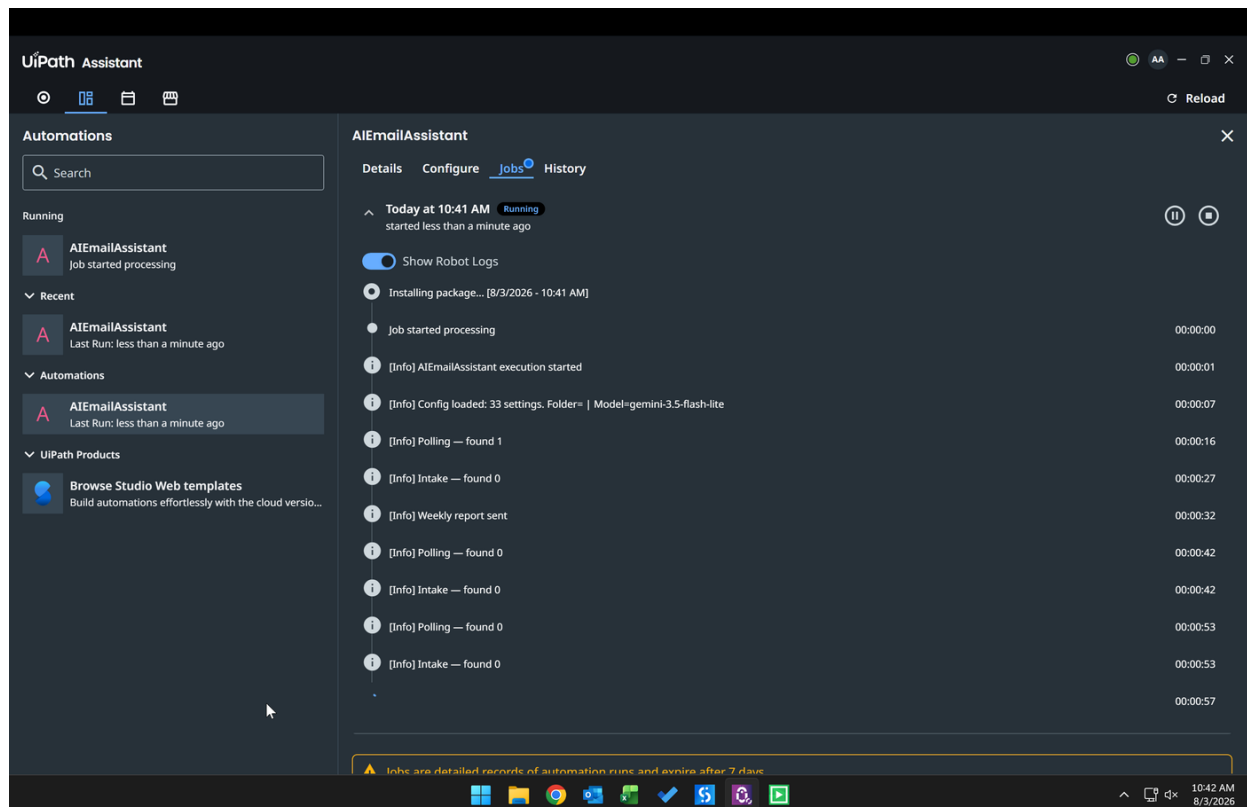


Figure 19. A live run. Robot logs show configuration loading, polling cycles, folder intake, and scheduled report generation.

AIEmailAssistant Public

Unpin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

zunecs Refine extraction rules and fix mailbox path handling 5b65ff5 · 27 minutes ago 6 Commits

.project	Initial commit: AI Email Assistant v1	last week
.tmh	Initial commit: AI Email Assistant v1	last week
Config	Refine extraction rules and fix mailbox path handling	27 minutes ago
Documentation	Refine extraction rules and fix mailbox path handling	27 minutes ago
Workflows	Refine extraction rules and fix mailbox path handling	27 minutes ago
.gitignore	Add document processing features and clean up tracked r...	last week
Main.xaml	Add attachment extraction, library filing, and document int...	last week
README.md	Add document processing features and clean up tracked r...	last week
entry-points.json	Initial commit: AI Email Assistant v1	last week
project.json	Add attachment extraction, library filing, and document int...	last week
project.uiproj	Initial commit: AI Email Assistant v1	last week

README

AI-Powered Email Assistant

UiPath RPA solution that monitors Outlook, extracts action items using Google Gemini, and creates calendar events, Microsoft To Do tasks, and draft replies automatically. Also files document attachments into a categorised library with AI-generated summaries.

Built as a summer COOP internship project at Saudi Aramco, 2026.

About

UiPath RPA solution that reads Outlook emails and documents, extracts action items with Google Gemini, and creates calendar events, To Do tasks, draft replies, and a categorised document library.

automation document-processing gemini-api internship-project ilm microsoft365 outlook-automation rpa uipath vb-net

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

zunecs Alaamri, Abdulrahman S.

Figure 20. The public repository. Configuration secrets and generated runtime data are excluded through .gitignore; a redacted template documents the required settings.

12. Limitations

The following constraints are known and accepted at this version. Each is a consequence of a documented decision rather than an oversight.

Limitation	Cause and consequence
Windows-only execution	COM automation binds the solution to Windows with classic Outlook installed, signed in, and running. It cannot run headless.
Attended operation	Unattended scheduling requires a licensed orchestration tier. The process runs under a signed-in session, started manually or by the operating system scheduler.
Vector cache reflects creation, not existence	Duplicate detection compares against tasks the robot created. A task deleted manually will not be recreated until its cache entry ages out.
Report state is not persisted	The last-report marker is held in memory. A restart on the report day after the trigger hour would produce a second report.
Replies are not threaded	The mail object does not expose the identifier required for a true threaded reply, so drafts are composed as new messages with a conventional subject prefix.
Scanned documents are not read	Image-based PDFs yield no text and are processed as though empty, without an error.
Shared attachment budget	The text ceiling applies to all attachments combined, so a long first document can crowd out a shorter later one.
Free-tier quotas	Request rates are subject to provider limits. Batch caps mitigate this but do not remove it.

13. Future Work

Enhancement	Value
Confidence-driven human review	Route low-confidence extractions to a review task rather than creating a calendar entry directly, making the system's own uncertainty visible to the user.
Conversation-thread awareness	Read earlier messages in a thread before extracting, so that follow-ups are interpreted against what was already agreed.
Direct task-store comparison	Query existing tasks rather than a local cache, removing the divergence between what the robot created and what currently exists.
Persistent report state	Write the last-report marker back to configuration so that scheduling survives a restart.
Commitment tracking	Apply the same extraction to sent mail, capturing promises the user made rather than requests they received.
Optical character recognition	Extend document intake to scanned material.
Unattended orchestration	Move to a licensed tier for continuous server-side execution independent of a signed-in session.

14. Setup Guide

14.1 Prerequisites

- Windows with UiPath Studio 2026 Community Edition installed.
- Classic Outlook desktop, signed in to a Microsoft Exchange account and running.
- Microsoft Word and Excel, required for Office attachment extraction.
- A Gemini API key, obtainable without charge from Google AI Studio.

14.2 Installation

9. Clone the repository and open project.json in UiPath Studio.
10. Copy Config/Config.template.json to Config/Config.json.
11. Enter the API key and the report recipient address in the new file.
12. Adjust the monitored folder, poll interval, and library paths as required.
13. In Outlook, create the three priority categories named AIEA-High, AIEA-Normal, and AIEA-Low, and create the processed-mail folder named in the configuration.
14. Run from Studio to verify, then publish and run from UiPath Assistant.

14.3 Operational notes

Situation	Action
Outlook is closed	Automation will start it, but the first cycle may be slow. Keeping Outlook open is recommended.
Tasks are not appearing	Confirm the account type is Exchange. Tasks on a local-only store do not synchronise to Microsoft To Do.
Categories appear without colour	The category does not yet exist in the mailbox. Create it once in Outlook.
A task is not recreated after deletion	Its entry remains in the vector cache. Delete the cache file to reset.
Repeated failures for one message	Inspect the audit log entries for that subject; the message remains in the monitored folder.

This document describes version 1.0 of the solution. All company names, personal names, and document contents shown in figures are fictional and were created for demonstration purposes.